Informe de Laboratorio 07

Tema: Desarrollo Backend con Express.js, Ruteo, Middlewares y Arquitectura por Capa

Nota

Estudiante	Escuela	Asignatura
Grupo 04 .	Escuela Profesional de Ingeniería de Sistemas	Introduccion al Desarrollo Web Semestre: II Código: 2025000

Laboratorio	Tema	Duración
07	Desarrollo Backend con Express.js, Ruteo, Middlewares y Arquitectura por Capas	04 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - B	Del 12 de octubre 2026	Al 19 de octubre 2026

1. Tarea

- Diseñar e implementar una API RESTful con Express.js, manejando los verbos HTTP (GET, POST, PUT, DELETE) para administrar recursos.
- Estructurar el proyecto aplicando una arquitectura por capas que separe las rutas, los controladores y la persistencia de datos.
- Implementar middlewares incorporados y personalizados para procesar solicitudes JSON, registrar actividad (logs) y manejar errores.
- Probar y documentar el correcto funcionamiento de los endpoints mediante una colección de pruebas en Postman o Thunder Client.

2. Equipos, materiales y temas utilizados

- Visual Studio Code 1.137.0 .
- Thunder Client v2.41.3.
- Express.js, coleccion-postman-cursos.json, JavaScript.
- GitHub con cuenta institucional.
- Terminal de comandos .
- Git 2.55.0.windows.3
- Node.js version 24.21.0 y npm.

3. URL de Repositorio Github

- URL del Repositorio GitHub para clonar o recuperar.
- https://github.com/ppachao-cmyk/ctx.git

4. Actividades del proyecto

4.1. Paso 1: Estructura del repositorio con backend, bd y frontend

4.1.1. Creamos las carpetas y subcarpetas

```
git add .gitignore backend/.gitignore bd/.gitignore frontend/.gitignore
    backend/README.md bd/README.md frontend/README.md frontend/priv/.gitkeep
    frontend/pub/img/.gitkeep frontend/pub/fonts/.gitkeep
```

4.2. Paso 2: Proyecto npm con express, express-session, bcryptjs y dotenv

4.2.1. Comandos en la terminal

```
cd backend
npm init -y
npm install express express-session bcryptjs dotenv
```

4.2.2. Modificar backend/package.json

```
{
  "name": "unimarket",
  "version": "1.0.0",
  "description": "Marketplace universitario con registro, sesiones seguras y
    resenas (Laboratorio 07)",
  "main": "app.js",
  "scripts": {
    "start": "node app.js",
    "dev": "node --watch app.js"
  },
  "author": "",
  "license": "ISC",
  "dependencies": {
    "bcryptjs": "^3.0.3",
    "dotenv": "^18.0.7",
    "express": "^5.3.0",
    "express-session": "^1.19.0"
  }
}
```

4.2.3. Crear plantilla backend/.env.example

```
PORT=3000
SESSION_SECRET=cambia_esto_por_una_clave_larga_y_aleatoria
```

4.3. Paso 3: Variables de entorno con dotenv y servidor base en el puerto del .env

4.3.1. Crear archivo local backend/.env

```
PORT=3000
SESSION_SECRET=clave_secreta_super_segura
```

4.3.2. Código del servidor en backend/app.js

```
require('dotenv').config({ quiet: true });
const express = require('express');[cite: 1]
const app = express();[cite: 1]
const PORT = process.env.PORT || 3000;[cite: 1]

app.use(express.json());[cite: 1]

app.get('/', (req, res) => {[cite: 1]
  res.json({ message: 'UniMarket funcionando' });[cite: 1]
});[cite: 1]

app.listen(PORT, () => console.log(`UniMarket escuchando en
    http://localhost:${PORT}`));[cite: 1]
```

El código presenta la implementación de un servidor básico utilizando Node.js y Express

- Se importa dotenv para la gestión de variables de entorno y se instancia el módulo express en la constante app, que actuará como el núcleo del servidor.
- Se define el puerto de escucha (PORT), asignando el valor de las variables de entorno o, por defecto, el puerto 3000. Adicionalmente, se habilita el middleware express.json() para el procesamiento de datos en formato JSON.
- Se establece un endpoint de tipo GET en la ruta raíz (/). Al ser consultado, el servidor retorna una respuesta en formato JSON con el mensaje: 'UniMarket funcionando'.
- Mediante app.listen, el servidor se inicializa en el puerto designado e imprime en consola la URL de acceso local.

4.4. Paso 4: Registro de usuarios con hash

4.4.1. Persistencia de usuarios

```
const usuarios = [];[cite: 1]
module.exports = usuarios;[cite: 1]
```

Para poder almacenar los datos, se declara una constante llamada usuarios inicializada como un arreglo vacío ([]), el cual funciona como una base de datos en memoria RAM. Luego, para permitir que otros archivos del proyecto accedan a esta información, se exporta el arreglo utilizando module.exports.

4.4.2. Utilidades de apoyo

```
// utils/siguienteId.js
function siguienteId(arreglo) {
  return arreglo.length > 0 ? Math.max(...arreglo.map((elemento) =>
    elemento.id)) + 1 : 1;[cite: 1]
}
module.exports = siguienteId;[cite: 1]

// utils/usuarioPublico.js
function usuarioPublico(usuario) {
  return { id: usuario.id, nombre: usuario.nombre, email: usuario.email
    };[cite: 1]
}
module.exports = usuarioPublico;[cite: 1]
```

Para poder generar identificadores únicos, definimos la función siguienteId, la cual evalúa un arreglo: si está vacío retorna 1, y si contiene elementos, calcula el valor máximo de los IDs existentes y le suma 1. Por otro lado, para proteger la información sensible, creamos la función usuarioPublico, que recibe un objeto usuario y retorna uno nuevo con campos seleccionados (id, nombre y email), omitiendo datos privados. Finalmente, exportamos ambas funciones mediante module.exports para su uso en otras partes del proyecto.

4.4.3. Controlador de autenticación

```
const bcrypt = require('bcryptjs');[cite: 1]
const usuarios = require('../data/usuarios');[cite: 1]
const siguienteId = require('../utils/siguienteId');[cite: 1]
const usuarioPublico = require('../utils/usuarioPublico');[cite: 1]
const SALT_ROUNDS = 10;[cite: 1]

exports.register = async (req, res) => {
  const { nombre, email, password } = req.body ?? {};[cite: 1]
  if (typeof nombre !== 'string' || typeof email !== 'string' || typeof
    password !== 'string'
      || !nombre || !email || !password) {[cite: 1]
    return res.status(400).json({ status: 'fail', message: 'Faltan campos
    obligatorios: nombre, email y password' });[cite: 1]
  }

  const passwordHash = await bcrypt.hash(password, SALT_ROUNDS);[cite: 1]

  if (usuarios.some((u) => u.email === email)) {[cite: 1]
    return res.status(409).json({ status: 'fail', message: 'Ese email ya áest
    registrado' });[cite: 1]
  }

  const nuevoUsuario = { id: siguienteId(usuarios), nombre, email,
    passwordHash };[cite: 1]
  usuarios.push(nuevoUsuario);[cite: 1]
  res.status(201).json({ status: 'success', data:
    usuarioPublico(nuevoUsuario) });[cite: 1]
};
```

Para poder procesar el registro, importamos las librerías y módulos necesarios (bcryptjs, el arreglo de usuarios y las utilidades). Definimos la función asíncrona register, donde primero validamos que los campos nombre, email y password sean enviados y sean de tipo texto; si faltan, retornamos un error 400. Si los datos son correctos, encriptamos la contraseña usando bcrypt y verificamos si el email ya existe en el arreglo; de ser así, retornamos un error 409. Finalmente, creamos el nuevo objeto usuario con un ID autogenerado y la contraseña encriptada, lo añadimos a la lista y respondemos con un estado 201 y los datos públicos del usuario.

4.4.4. Rutas e integración

```
// routes/authRoutes.js
const express = require('express');[cite: 1]
const router = express.Router();[cite: 1]
const authController = require('../controllers/authController');[cite: 1]

router.post('/register', authController.register);[cite: 1]
module.exports = router;[cite: 1]

// app.js
require('dotenv').config({ quiet: true });[cite: 1]
const express = require('express');[cite: 1]
const authRoutes = require('./routes/authRoutes');[cite: 1]
const app = express();[cite: 1]
const PORT = process.env.PORT || 3000;[cite: 1]

app.use(express.json());[cite: 1]
app.use('/api/auth', authRoutes);[cite: 1]
app.listen(PORT, () => console.log(`UniMarket escuchando en
    http://localhost:${PORT}`));[cite: 1]
```

Para poder gestionar las peticiones de autenticación, creamos un enrutador con express.Router() e importamos el controlador de autenticación definido previamente. Configuramos una ruta de tipo POST en /register que ejecuta la función register de dicho controlador, exportando este enrutador para su uso externo. Posteriormente, en el archivo principal app.js, importamos este enrutador y lo vinculamos a la aplicación mediante app.use, asignándole el prefijo /api/auth.

4.5. Paso 5: Login y sesión

4.5.1. Constantes del sistema

```
module.exports = {
  NOMBRE_COOKIE: 'compratux.sid',
};
```

Para poder centralizar la configuración, exportamos un objeto mediante module.exports que contiene la constante NOMBRE-COOKIE, a la cual le asignamos el valor 'compratux.sid.'

4.5.2. Controlador de login

```
exports.login = async (req, res) => {
  const { email, password } = req.body ?? {};
  if (typeof email !== 'string' || typeof password !== 'string' || !email ||
    !password) {
    return res.status(400).json({ status: 'fail', message: 'Debes enviar
    email y password' });
  }

  const usuario = usuarios.find((u) => u.email === email);
  if (!usuario) {
    return res.status(401).json({ status: 'fail', message: 'Credenciales
    áinvlidas' });
  }

  const coincide = await bcrypt.compare(password, usuario.passwordHash);
  if (!coincide) {
    return res.status(401).json({ status: 'fail', message: 'Credenciales
    áinvlidas' });
  }

  req.session.userId = usuario.id;
  res.status(200).json({ status: 'success', data: usuarioPublico(usuario) });
};
```

Para poder validar el acceso, definimos la función asíncrona login, donde primero verificamos que el email y la contraseña hayan sido enviados y sean de tipo texto; si faltan o son incorrectos, retornamos un error 400. Luego, buscamos al usuario en el arreglo por su email y, si no lo encontramos, devolvemos un error 401. Si el usuario existe, comparamos la contraseña enviada con la encriptada usando bcrypt.compare; si no coinciden, retornamos el mismo error 401. Finalmente, si las credenciales son válidas, guardamos el ID del usuario en la sesión (req.session.userId) y respondemos con un estado 200 junto a los datos públicos del usuario.

4.5.3. Definición de rutas

```
const express = require('express');
const router = express.Router();
const authController = require('../controllers/authController');

router.post('/register', authController.register);
router.post('/login', authController.login);

module.exports = router;
```

Para poder gestionar las peticiones web, importamos Express e inicializamos un enrutador con express.Router(). Luego, importamos el controlador de autenticación y definimos dos rutas de tipo POST: una dirigida a /register que ejecuta la función register, y otra hacia /login que ejecuta la función login. Finalmente, exportamos el enrutador mediante module.exports para que sea integrado en el servidor principal.

4.5.4. Configuración del servidor y middleware de sesión

```
require('dotenv').config({ quiet: true });
const express = require('express');
const session = require('express-session');
const { NOMBRE_COOKIE } = require('./config/constantes');
const authRoutes = require('./routes/authRoutes');

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());
app.use(
  session({
    name: NOMBRE_COOKIE,
    secret: process.env.SESSION_SECRET,
    resave: false,
    saveUninitialized: false,
    cookie: {
      httpOnly: true,
      secure: process.env.NODE_ENV === 'production',
      maxAge: 1000 * 60 * 60 * 2,
    },
  })
);

app.use('/api/auth', authRoutes);

app.listen(PORT, () => console.log(`UniMarket escuchando en
    http://localhost:${PORT}`));
```

Para poder inicializar el servidor, configuramos las variables de entorno e importamos las dependencias necesarias, incluyendo express-session, nuestras constantes y las rutas de autenticación. Instanciamos la aplicación (app), definimos el puerto de escucha y habilitamos el middleware express.json() para procesar datos entrantes. Posteriormente, configuramos el middleware de sesión asignándole el nombre de nuestra cookie, una clave secreta desde las variables de entorno y opciones de seguridad como httpOnly: true, secure en producción y un tiempo de expiración de dos horas. Finalmente, vinculamos las rutas de autenticación bajo el prefijo /api/auth e iniciamos el servidor con app.listen.

4.6. Paso 6: AuthMiddleware y ruta privada

4.6.1. Middleware de autenticación

```
function authMiddleware(req, res, next) {
  if (req.session && req.session.userId) {
    return next();
  }
  return res.status(401).json({
    status: 'fail',
    message: 'Acceso no autorizado: inicia ósesin primero',
  });
}

```

```
module.exports = authMiddleware;
```

Para poder restringir el acceso a los usuarios no autenticados, definimos la función authMiddleware, la cual recibe los parámetros req, res y next. Dentro de la función, verificamos si existe una sesión activa comprobando req.session y si contiene el userId. Si la sesión es válida, permitimos que la petición continúe su curso llamando a next(). En caso contrario, detenemos la ejecución y retornamos una respuesta con estado 401, indicando que el acceso no está autorizado y solicitando el inicio de sesión. Finalmente, exportamos esta función mediante module.exports para que pueda ser utilizada en otras rutas.

4.6.2. Controlador de perfil

```
const usuarios = require('../data/usuarios');
const usuarioPublico = require('../utils/usuarioPublico');

exports.getPerfil = (req, res) => {
  const usuario = usuarios.find((u) => u.id === req.session.userId);
  if (!usuario) {
    return res.status(401).json({ status: 'fail', message: 'Acceso no
    autorizado: inicia ósesin primero' });
  }
  res.status(200).json({ status: 'success', data: usuarioPublico(usuario) });
};
```

Para poder entregar la información del perfil, importamos el arreglo de usuarios y la utilidad usuarioPublico. Definimos la función getPerfil, donde buscamos al usuario en la lista comparando su id con el userId almacenado en la sesión. Si no encontramos coincidencias, retornamos un error 401 indicando que se requiere iniciar sesión. Si el usuario existe, respondemos con un estado 200 y enviamos sus datos a través de usuarioPublico para omitir información sensible como la contraseña.

4.6.3. Definición de rutas

```
const express = require('express');
const router = express.Router();
const perfilController = require('../controllers/perfilController');
const authMiddleware = require('../middlewares/authMiddleware');

router.get('/', authMiddleware, perfilController.getPerfil);

module.exports = router;
```

Para poder gestionar la consulta del perfil, inicializamos un enrutador con express.Router() e importamos el controlador de perfil y el middleware de autenticación. Definimos una ruta de tipo GET en la raíz (/), a la cual le aplicamos el authMiddleware antes de ejecutar perfilController.getPerfil. Esto garantiza que la petición pase primero por el filtro de seguridad. Finalmente, exportamos el enrutador mediante module.exports.

4.6.4. Integración en app.js

```
require('dotenv').config({ quiet: true });
const express = require('express');
const session = require('express-session');
const { NOMBRE_COOKIE } = require('./config/constantes');
const authRoutes = require('./routes/authRoutes');
const perfilRoutes = require('./routes/perfilRoutes');

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());
app.use(
  session({
    name: NOMBRE_COOKIE,
    secret: process.env.SESSION_SECRET,
    resave: false,
    saveUninitialized: false,
    cookie: {
      httpOnly: true,
      secure: process.env.NODE_ENV === 'production',
      maxAge: 1000 * 60 * 60 * 2,
    },
  })
);

app.use('/api/auth', authRoutes);
app.use('/api/perfil', perfilRoutes);

app.listen(PORT, () => console.log(`UniMarket escuchando en
    http://localhost:${PORT}`));
```

Para poder centralizar la aplicación, importamos las dependencias base (incluyendo express-session), las constantes del sistema y los enrutadores de autenticación y perfil. Inicializamos la aplicación (app), definimos el puerto de escucha y habilitamos el middleware express.json(). Posteriormente, configuramos el middleware de sesión con opciones de seguridad (como httpOnly, secure en producción y tiempo de expiración). Finalmente, vinculamos las rutas bajo los prefijos /api/auth y /api/perfil mediante app.use, e iniciamos el servidor con app.listen.

4.7. Paso 7: Logout que destruye la sesion y limpia la cookie

4.7.1. Actualización de importaciones en controllers/authController.js

```
const { NOMBRE_COOKIE } = require('../config/constantes');
```

Para poder utilizar el nombre estandarizado de la cookie al momento de cerrar sesión, importamos la constante NOMBRE-COOKIE desde el archivo de configuración mediante la desestructuración de require.

4.7.2. Controlador de logout en controllers/authController.js

```
exports.logout = (req, res, next) => {
  req.session.destroy((err) => {
    if (err) return next(err);
    res.clearCookie(NOMBRE_COOKIE);
    res.status(200).json({ status: 'success', message: 'óSesin cerrada
    correctamente' });
  });
};
```

Para poder finalizar la sesión activa, definimos la función logout, la cual utiliza req.session.destroy para eliminar los datos de la sesión en el servidor. Si ocurre un error durante este proceso, lo pasamos al manejador de errores mediante next(err). Si la destrucción es exitosa, limpiamos la cookie del navegador usando res.clearCookie(NOMBRE-COOKIE) y finalmente respondemos con un estado 200 y un mensaje de éxito.

4.7.3. Definición de rutas en routes/authRoutes.js

```
const express = require('express');
const router = express.Router();
const authController = require('../controllers/authController');
const authMiddleware = require('../middlewares/authMiddleware');

router.post('/register', authController.register);
router.post('/login', authController.login);
router.post('/logout', authMiddleware, authController.logout);

module.exports = router;
```

Para poder gestionar las peticiones de autenticación, inicializamos un enrutador con express.Router() e importamos el controlador y el middleware de autenticación. Definimos tres rutas de tipo POST: /register y /login que ejecutan sus respectivos controladores directamente, y /logout a la cual le añadimos el authMiddleware antes de llamar al controlador de cierre de sesión. Finalmente, exportamos el enrutador mediante module.exports.

4.8. Paso 8: Publicar productos con sesion y consultarlos de forma publica

4.8.1. Persistencia de datos en backend/data/

```
// data/productos.js
const productos = [];
module.exports = productos;

// data/resenas.js
const resenas = [];
module.exports = resenas;
```

Para poder almacenar la información temporalmente, definimos en data/productos.js y data/resenas.js arreglos vacíos (productos = [] y resenas = []) y los exportamos mediante module.exports. Por otro lado, para poder procesar datos numéricos de forma segura, creamos la función aNumero en utils/aNumero.js. Esta función recibe un valor y verifica su tipo: si ya es un número lo retorna directamente; si es un texto (string) sin espacios vacíos, lo convierte a número; y si no cumple estas condiciones, retorna NaN.

4.8.2. CUtilidad de conversión en utils/aNumero.js

```
function aNumero(valor) {
  if (typeof valor === 'number') return valor;
  if (typeof valor === 'string' && valor.trim() !== '') return Number(valor);
  return NaN;
}
module.exports = aNumero;
```

Para poder procesar datos numéricos de forma segura, definimos la función aNumero, la cual recibe un valor y evalúa su tipo. Si el valor ya es un número (typeof valor === 'number'), lo retornamos directamente. Si es un texto (string), verificamos que no esté vacío mediante trim(), y de ser así, lo convertimos a número usando Number(). Si no cumple ninguna de estas condiciones, retornamos NaN. Finalmente, exportamos la función mediante module.exports.

4.8.3. Controlador de productos en controllers/productoController.js

```
const productos = require('../data/productos');
const usuarios = require('../data/usuarios');
const resenas = require('../data/resenas');
const siguienteId = require('../utils/siguienteId');
const aNumero = require('../utils/aNumero');

function presentar(producto) {
  const vendedor = usuarios.find((u) => u.id === producto.vendedorId);
  const delProducto = resenas.filter((r) => r.productoId === producto.id);
  const suma = delProducto.reduce((acumulado, r) => acumulado +
    r.calificacion, 0);
  return {
    ...producto,
    vendedor: vendedor ? vendedor.nombre : 'Usuario eliminado',
    promedio: delProducto.length > 0 ? Math.round((suma / delProducto.length)
    * 10) / 10 : null,
    totalResenas: delProducto.length,
  };
}

function limpiarCampos(body, exigirTodos) {
  const { nombre, descripcion, precio, categoria, stock } = body ?? {};
  const limpio = {};
  if (exigirTodos || nombre !== undefined) {
    if (typeof nombre !== 'string' || nombre.trim().length < 3 ||
    nombre.trim().length > 80) {
      return { error: 'El nombre debe tener entre 3 y 80 caracteres' };
    }
```

```
26      limpio.nombre = nombre.trim();
27    }
28    if (exigirTodos || descripcion !== undefined) {
29      if (typeof descripcion !== 'string' || !descripcion.trim() ||
        descripcion.trim().length > 500) {
30        return { error: 'La ódescripcin es obligatoria (ámximo 500 caracteres)'
        };
31      }
32      limpio.descripcion = descripcion.trim();
33    }
34    if (exigirTodos || precio !== undefined) {
35      const valor = aNumero(precio);
36      if (!Number.isFinite(valor) || valor <= 0) {
37        return { error: 'El precio debe ser un únmero mayor a 0' };
38      }
39      limpio.precio = Math.round(valor * 100) / 100;
40    }
41    if (exigirTodos || categoria !== undefined) {
42      if (typeof categoria !== 'string' || !categoria.trim() ||
        categoria.trim().length > 30) {
43        return { error: 'La ícategora es obligatoria (ámximo 30 caracteres)' };
44      }
45      limpio.categoria = categoria.trim();
46    }
47    if (exigirTodos || stock !== undefined) {
48      const valor = aNumero(stock);
49      if (!Number.isInteger(valor) || valor < 0) {
50        return { error: 'El stock debe ser un únmero entero mayor o igual a 0'
        };
51      }
52      limpio.stock = valor;
53    }
54    return { limpio };
55  }
56
57  exports.listarProductos = (req, res) => {
58    const { categoria, buscar, vendedor } = req.query;
59    let resultado = productos;
60    if (categoria) {
61      resultado = resultado.filter((p) => p.categoria.toLowerCase() ===
        String(categoria).toLowerCase());
62    }
63    if (buscar) {
64      const texto = String(buscar).toLowerCase();
65      resultado = resultado.filter(
66        (p) => p.nombre.toLowerCase().includes(texto) ||
        p.descripcion.toLowerCase().includes(texto)
67      );
68    }
69    if (vendedor) {
70      resultado = resultado.filter((p) => p.vendedorId === Number(vendedor));
71    }
72    const ordenados = [...resultado].reverse().map(presentar);
73    res.status(200).json({ status: 'success', results: ordenados.length, data:
        ordenados });
74  };
```

```
75
76 exports.obtenerProducto = (req, res) => {
77   const producto = productos.find((p) => p.id === Number(req.params.id));
78   if (!producto) {
79     return res.status(404).json({ status: 'fail', message: 'Producto no
     encontrado' });
80   }
81   res.status(200).json({ status: 'success', data: presentar(producto) });
82 };
83
84 exports.crearProducto = (req, res) => {
85   const { error, limpio } = limpiarCampos(req.body, true);
86   if (error) {
87     return res.status(400).json({ status: 'fail', message: error });
88   }
89   const nuevoProducto = {
90     id: siguienteId(productos),
91     ...limpio,
92     vendedorId: req.session.userId,
93     creadoEn: new Date().toISOString(),
94   };
95   productos.push(nuevoProducto);
96   res.status(201).json({ status: 'success', data: presentar(nuevoProducto) });
97 };
```

Para poder presentar los productos con información enriquecida, definimos la función presentar, la cual busca el nombre del vendedor y calcula el promedio y total de sus reseñas. Asimismo, para validar los datos entrantes, creamos la función limpiarCampos, que verifica los tipos de datos y los límites de longitud, además de convertir valores numéricos usando la utilidad aNumero.

En cuanto a los controladores exportados: listarProductos filtra el catálogo por categoría, búsqueda de texto o vendedor, devolviendo la lista ordenada. obtenerProducto busca un producto por su ID y retorna un error 404 si no existe. crearProducto valida los campos obligatorios, asigna el ID con siguienteId y extrae el vendedorId directamente de la sesión (req.session.userId) por seguridad. actualizarProducto permite modificaciones parciales solo si el usuario en sesión es el dueño del producto (retorna 403 si no lo es). Finalmente, eliminarProducto verifica la propiedad del producto, lo remueve del arreglo y, además, elimina en cascada todas las reseñas asociadas a ese producto.

4.8.4. Definición de rutas e integración en outes/productoRoutes.js y app.js

```
1  // routes/productoRoutes.js
2  const express = require('express');
3  const router = express.Router();
4  const productoController = require('../controllers/productoController');
5  const authMiddleware = require('../middlewares/authMiddleware');
6
7  router.get('/', productoController.listarProductos);
8  router.get('/:id', productoController.obtenerProducto);
9  router.post('/', authMiddleware, productoController.crearProducto);
10
11 module.exports = router;
```

```
12
13 // app.js
14 require('dotenv').config({ quiet: true });
15 const express = require('express');
16 const session = require('express-session');
17 const { NOMBRE_COOKIE } = require('./config/constantes');
18 const authRoutes = require('./routes/authRoutes');
19 const perfilRoutes = require('./routes/perfilRoutes');
20 const productoRoutes = require('./routes/productoRoutes');
21
22 const app = express();
23 const PORT = process.env.PORT || 3000;
24
25 app.use(express.json());
26 app.use(
27   session({
28     name: NOMBRE_COOKIE,
29     secret: process.env.SESSION_SECRET,
30     resave: false,
31     saveUninitialized: false,
32     cookie: {
33       httpOnly: true,
34       secure: process.env.NODE_ENV === 'production',
35       maxAge: 1000 * 60 * 60 * 2,
36     },
37   })
38 );
39
40 app.use('/api/auth', authRoutes);
41 app.use('/api/perfil', perfilRoutes);
42 app.use('/api/productos', productoRoutes);
43
44 app.listen(PORT, () => console.log(`UniMarket escuchando en
    http://localhost:${PORT}`));
```

Para poder gestionar las peticiones de productos, en productoRoutes.js inicializamos un enrutador e importamos el controlador y el middleware de autenticación. Definimos rutas públicas de tipo GET para listar (/) y obtener un producto por ID (/:id). Asimismo, configuramos una ruta POST (/) protegida con authMiddleware para crear productos, y finalmente exportamos el enrutador.

Posteriormente, en app.js importamos este nuevo enrutador junto con los anteriores. Tras configurar la aplicación, el procesamiento de JSON y el middleware de sesión con sus opciones de seguridad, vinculamos las rutas de productos bajo el prefijo /api/productos mediante app.use.

4.9. Paso 9: Editar y eliminar productos solo por su dueño

4.9.1. Controlador de actualización y eliminación en controllers/productoController.js

```
1 exports.actualizarProducto = (req, res) => {
2   const producto = productos.find((p) => p.id === Number(req.params.id));
3   if (!producto) {
```

```
    return res.status(404).json({ status: 'fail', message: 'Producto no encontrado' });
  }
  if (producto.vendedorId !== req.session.userId) {
    return res.status(403).json({ status: 'fail', message: 'Solo el vendedor puede modificar este producto' });
  }
  const { error, limpio } = limpiarCampos(req.body, false);
  if (error) {
    return res.status(400).json({ status: 'fail', message: error });
  }
  if (Object.keys(limpio).length === 0) {
    return res.status(400).json({ status: 'fail', message: 'No enviaste úningn campo para actualizar' });
  }
  Object.assign(producto, limpio);
  res.status(200).json({ status: 'success', data: presentar(producto) });
};

exports.eliminarProducto = (req, res) => {
  const id = Number(req.params.id);
  const indice = productos.findIndex((p) => p.id === id);
  if (indice === -1) {
    return res.status(404).json({ status: 'fail', message: 'Producto no encontrado' });
  }
  if (productos[indice].vendedorId !== req.session.userId) {
    return res.status(403).json({ status: 'fail', message: 'Solo el vendedor puede eliminar este producto' });
  }
  productos.splice(indice, 1);
  for (let i = resenas.length - 1; i >= 0; i--) {
    if (resenas[i].productoId === id) resenas.splice(i, 1);
  }
  res.status(200).json({ status: 'success', message: 'Producto eliminado correctamente' });
};
```

4.9.2. Definición de rutas en routes/productoRoutes.js

```
const express = require('express');
const router = express.Router();
const productoController = require('../controllers/productoController');
const authMiddleware = require('../middlewares/authMiddleware');

router.get('/', productoController.listarProductos);
router.get('/:id', productoController.obtenerProducto);
router.post('/', authMiddleware, productoController.crearProducto);
router.put('/:id', authMiddleware, productoController.actualizarProducto);
router.delete('/:id', authMiddleware, productoController.eliminarProducto);

module.exports = router;
```

4.10. Paso 10: compra simulada

4.10.1. Datos en memoria (data/compras.js)

```
const compras = [];
module.exports = compras;
```

4.10.2. Validaciones al comprar (compraController.js)

```
if (!producto) return res.status(404)...                    // no existe
if (producto.vendedorId === req.session.userId) return res.status(403)...  //
    es tuyo
if (!Number.isInteger(cantidad) || cantidad < 1) return res.status(400)... //
    cantidad áinvlida
if (producto.stock < cantidad) return res.status(409)...     // sin stock
```

4.10.3. Descontar stock y guardar la compra

```
producto.stock -= cantidad;
const compra = {
  id: siguienteId(compras),
  productoId: producto.id,
  productoNombre: producto.nombre,     // copia, por si el producto cambia o
    se borra
  precioUnitario: producto.precio,
  cantidad,
  total: Math.round(producto.precio * cantidad * 100) / 100, // lo calcula el
    servidor
  vendedorId: producto.vendedorId,
  compradorId: req.session.userId,
  fecha: new Date().toISOString(),
};
compras.push(compra);
```

4.10.4. Mis compras y mis ventas (mismo controlador, filtran por usuario)

```
compras.filter((c) => c.compradorId === req.session.userId)  // misCompras
compras.filter((c) => c.vendedorId === req.session.userId)   // misVentas
```

4.10.5. Rutas (compraRoutes.js y productoRoutes.js)

```
router.get('/compras', authMiddleware, compraController.misCompras);
router.get('/ventas',  authMiddleware, compraController.misVentas);
// en productoRoutes:
router.post('/:id/comprar', authMiddleware, compraController.comprar);
```

4.10.6. Montar en app.js

```
app.use('/api', compraRoutes);
```

4.11. Paso 11: reseñas

4.11.1. Validaciones al crear una reseña (resenaController.js)

```
if (producto.vendedorId === req.session.userId) return res.status(403)... // no ñreseas lo tuyo
if (!Number.isInteger(puntaje) || puntaje < 1 || puntaje > 5) return res.status(400)...
if (resenas.some((r) => r.productoId === id && r.usuarioId === req.session.userId))
  return res.status(409)...  // ya ñreseaste
```

4.11.2. Guardar la reseña

```
app.use('/api', compraRoutes);
```

4.11.3. Listar reseñas con el nombre del autor (pública)

```
const nuevaResena = {
  id: siguienteId(resenas),
  productoId: id,
  usuarioId: req.session.userId,   // sale de la ósesin, nunca del body
  calificacion: puntaje,
  comentario: comentario.trim(),
  fecha: new Date().toISOString(),
};
resenas.push(nuevaResena);
```

4.11.4. Eliminar solo si eres el autor

```
resenas.filter((r) => r.productoId === id)
  .map((r) => ({ ...r, autor: autor ? autor.nombre : 'Usuario eliminado' }))
```

4.11.5. Rutas

```
if (resenas[indice].usuarioId !== req.session.userId) return res.status(403)...
resenas.splice(indice, 1);
```

4.11.6. Montar en app.js

```
// resenaRoutes.js (montado en /api/resenas)
router.delete('/:id', authMiddleware, resenaController.eliminarResena);
// productoRoutes.js
router.get('/:id/resenas',  resenaController.listarPorProducto);
router.post('/:id/resenas', authMiddleware, resenaController.crearResena);
```

4.12. Paso 12: esquema SQL (bd/bd.sql)

4.12.1. Tablas base

```
CREATE TABLE usuarios (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  email VARCHAR(255) NOT NULL UNIQUE,
  password_hash VARCHAR(60) NOT NULL ...
);
```

4.12.2. Reglas dentro de la tabla

```
precio NUMERIC(10, 2) NOT NULL CHECK (precio > 0),
stock INTEGER NOT NULL CHECK (stock >= 0),
FOREIGN KEY (vendedor_id) REFERENCES usuarios(id)
```

4.12.3. Compras conservan el historial

```
producto_id INTEGER,   -- puede quedar NULL
FOREIGN KEY (producto_id) REFERENCES productos(id) ON DELETE SET NULL
```

4.12.4. Reseñas: una por persona y producto

```
calificacion INTEGER NOT NULL CHECK (calificacion BETWEEN 1 AND 5),
UNIQUE (producto_id, usuario_id),
FOREIGN KEY (producto_id) REFERENCES productos(id) ON DELETE CASCADE
```

4.13. Paso 13:la página, primera parte

4.13.1. Servir el frontend (app.js)

```
const FRONTEND = path.join(__dirname, '..', 'frontend');
app.use('/pub', express.static(path.join(FRONTEND, 'pub')));
app.get('/', (req, res) => res.sendFile(path.join(FRONTEND, 'index.html')));
```

4.13.2. Vistas ocultas en index.html (solo se muestra una a la vez)

```
<section id="vista-productos" class="vista">...</section>
<section id="vista-detalle"   class="vista oculto">...</section>
<section id="vista-auth"      class="vista oculto">...</section>
```

4.13.3. CSS con la paleta UNSA (style.css)

```
:root { --granate: #8E1B2E; --oro: #C8962E; --niebla: #EDF1F4; }
.oculto { display: none !important; }
```

4.13.4. Función api() que lanza error si la API falla (main.js, sección 1)

```
const respuesta = await fetch(url, opciones);
if (!respuesta.ok) {
  const fallo = new Error(json.message || 'óOcurri un error inesperado');
  throw fallo;
}
```

4.13.5. Cambiar de vista (sección 2)

```
if ((nombre === 'vender' || nombre === 'cuenta') && !usuarioActual) {
  mostrarVista('auth');   // pide login primero
  return;
}
```

4.13.6. Login y sesión activa (sección 3)

```
const json = await api('/api/auth/login', { method: 'POST', body: { email,
    password } });
usuarioActual = json.data;
// al abrir la ápgina:
const json = await api('/api/perfil');   // ¿ya hay ósesin?
```

4.13.7. Lista y búsqueda (sección 4)

```
const json = await api('/api/productos' + (consulta ? `?${consulta}` : ''));
dibujarProductos(listaProductos, json.data, null, 'íTodava no hay
    productos...');
```

4.14. Paso 14: vender y mi cuenta (main.js, secciones 5 y 6)

4.14.1. Un mismo formulario sirve para publicar y editar

```
const url    = editandoProductoId ? `/api/productos/${editandoProductoId}` :
    '/api/productos';
const metodo = editandoProductoId ? 'PUT' : 'POST';
await api(url, { method: metodo, body: datos });
```

4.14.2. Entrar en modo edición

```
editandoProductoId = producto.id;
document.querySelector('#prod-nombre').value = producto.nombre;
document.querySelector('#btn-guardar-producto').textContent = 'Guardar
    cambios';
```

4.14.3. Eliminar con confirmación

```
if (!confirm¿(`Seguro que quieres eliminar "${producto.nombre}"?...`)) return;
await api(`/api/productos/${producto.id}`, { method: 'DELETE' });
```

4.14.4. Cargar solo mis productos

```
const json = await api(`/api/productos?vendedor=${usuarioActual.id}`);
```

4.14.5. Compras y ventas en paralelo

```
const [compras, ventas] = await Promise.all([api('/api/compras'),
    api('/api/ventas')]);
```

4.15. Paso 15: detalle del producto (main.js, sección 7)

4.15.1. Abrir el detalle: producto y reseñas a la vez

```
const [producto, resenas] = await Promise.all([
  api(`/api/productos/${id}`),
  api(`/api/productos/${id}/resenas`),
]);
dibujarDetalle(producto.data, resenas.data);
```

4.15.2. El bloque de compra cambia según quién mira

```
if (esDueno) return el('p', {}, 'Este producto es tuyo...');
if (producto.stock === 0) return el('p', {}, 'Producto agotado');
if (!usuarioActual) return /* óbotn "Inicia ósesin para comprar" */;
// si no, el formulario con cantidad y óbotn Comprar
```

4.15.3. Comprar y refrescar el stock

```
const json = await api(`/api/productos/${producto.id}/comprar`, {
  method: 'POST', body: { cantidad: inputCantidad.value },
});
mostrarMensaje(`Compra realizada por ${formatoPrecio(json.data.total)}`);
abrirDetalle(producto.id);   // se redibuja con el stock nuevo
```

4.15.4. Dejar reseña (solo si corresponde)

```
const yaReseno = usuarioActual && resenas.some((r) => r.usuarioId ===
    usuarioActual.id);
if (esDueno) formulario = null;
else if (!usuarioActual) formulario = /* "Inicia ósesin para dejar una
    ñresea" */;
else if (yaReseno) formulario = /* "Ya dejaste tu ñresea" */;
else formulario = /* formulario con puntaje y comentario */;
```

4.15.5. Eliminar mi reseña

```
await api(`/api/resenas/${resenaId}`, { method: 'DELETE' });
abrirDetalle(productoId);
```

5. Producto Final

5.1. Estructura final

```
proyecto/
|-- backend
|   |-- confing
|       |-- constantes.js
|   |-- controllers
|       |-- authController.js
|       |-- compraController.js
|       |-- perfilController.js
|       |-- productoController.js
|       |-- resenaController.js
|   |-- data
|       |-- compras.js
|       |-- productos.js
|       |-- resenas.js
|       |-- usuarios.js
|   |-- middlewares
|       |-- authMiddleware.js
|   |-- routes
|       |-- authRoutes.js
|       |-- compraRoutes.js
|       |-- perfilRoutes.js
|       |-- productoRoutes.js
|       |-- resenaRoutes.js
|   |-- utils
|       |-- aNumero.js
|       |-- siguienteId.js
|       |-- usuarioPublico.js
|   |-- .env
|   |-- .env.example
|   |-- .gitignore
|   |-- app.js
|   |-- package-lock.json
|   |-- package.json
|   |-- README.md
|-- frontend
|   |-- .gitignore
|   |-- index.html
|   |-- pub
|        |-- img
|        |-- js
|           |-- main.js
|        |-- fonts
|        |-- style
|           |-- style.css
|   |-- priv
|   |-- index.html
|   |-- README.md
|-- .gitignore
```

```
|-- informes
|   |-- IDWeb_lab07_avance_proyecto.pdf

|-- README.md
```

6. Pruebas de trabajo grupal

6.1. Prueba 1

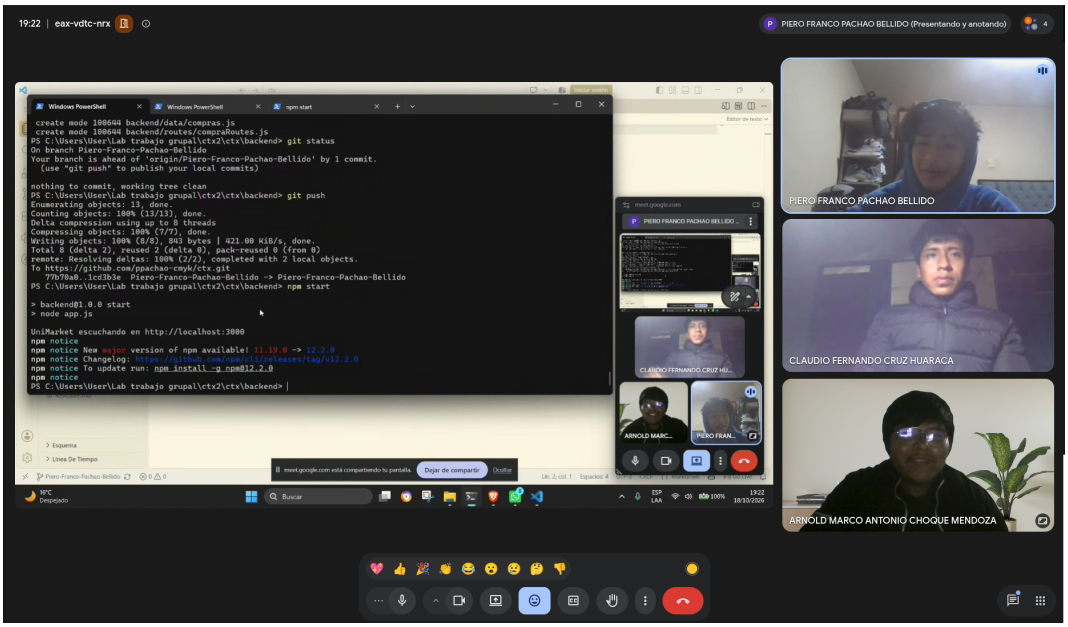


Figura 1: Evidencia de trabajo grupal

6.2. Prueba 2

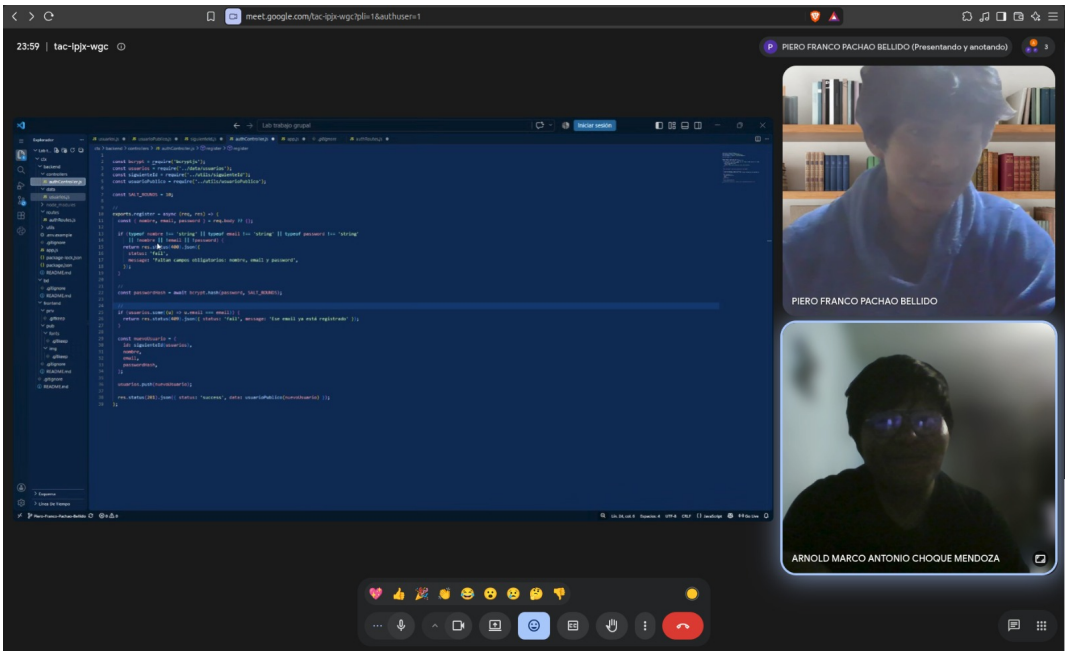


Figura 2: Evidencia de trabajo grupal

6.3. Prueba 3

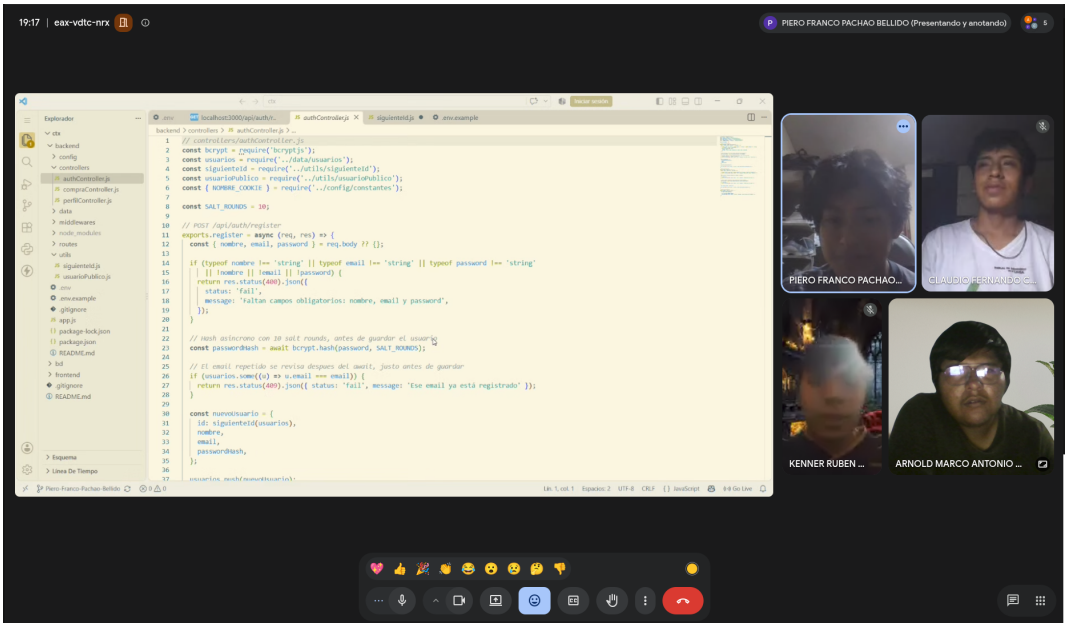


Figura 3: Evidencia de trabajo grupal

7. Preguntas

1. **¿Qué es un Middleware en Express y cuál es el impacto de no invocar la función next() dentro de su ejecución?**
   Un middleware es una función intermedia que procesa la información antes de que llegue a su destino final. Si **no se invoca next()**, la petición se queda bloqueada en el aire y el usuario nunca recibe una respuesta del servidor.

2. **Explique la diferencia de propósito semántico y de implementación entre req.params, req.query y req.body en Express.**

   - **req.params:** Sirve para capturar variables dinámicas integradas directamente en la ruta de la URL (ejemplo: `/cursos/:id`). Su propósito es identificar un recurso específico.
   - **req.query:** Sirve para capturar parámetros adicionales al final de la URL (ejemplo: `?creditos=4`). Su propósito es filtrar, buscar o aplicar configuraciones opcionales a una lista.
   - **req.body:** Sirve para recibir datos más complejos y ocultos en el cuerpo de la petición (generalmente en formato JSON). Su propósito es enviar la información necesaria para crear (POST) o actualizar (PUT) un recurso.

3. **¿Por qué es fundamental devolver códigos de estado HTTP semánticos (200, 201, 400, 404, 500) en una API RESTful en lugar de responder siempre con código 200 OK?**
   Porque **permiten que los sistemas clientes entiendan el resultado real al instante**. Un **201** confirma claramente que algo se guardó, un **404** advierte que no existe, y un **500** indica una falla en el servidor. Si siempre se enviara **200**, el cliente creería que todo salió bien y procesaría información equivocada.

8. Rúbricas

8.1. Entregable Informe

Tabla 1: Tipo de Informe

Informe	
Latex	El informe está en formato PDF desde Latex, con un formato limpio (buena presentación) y facil de leer.

8.2. Rúbrica para el contenido del Informe y demostración

- El alumno debe marcar o dejar en blanco en celdas de la columna **Checklist** si cumplio con el ítem correspondiente.
- Si un alumno supera la fecha de entrega, su calificación será sobre la nota mínima aprobada, siempre y cuando cumpla con todos lo items.
- El alumno debe autocalificarse en la columna **Estudiante** de acuerdo a la siguiente tabla:

Tabla 2: Rúbrica para contenido del Informe y demostración

Contenido y demostración		Puntos	Checklist	Estudiante	Profesor
1. Módulo de Autenticación y Hash	Implementa la autenticación completa. Cifra contraseñas obligatoriamente con bcryptjs y compara hashes de forma asíncrona impecablemente.	7	X	4	
2. Manejo de Sesiones y Protección.	Configura express-session con cookies httpOnly. Protege adecuadamente las rutas privadas mediante middlewares de autorización.	7	X	5	
3. Seguridad y Variables de Entorno.	Aisla completamente secretos en '.env'. No expone datos sensibles ni contraseñas en las respuestas JSON devueltas.	3	X	2	
4. Cuestionario Teórico.	Demuestra conocimiento amplio en vectores de ataque OWASP, hashing, salado y banderas de protección de cookies.	3	X	2	
Total		20		13	

9. Referencias

- Express.js Community. (2025). Express Routing and Middleware Documentation. Expressjs.com.
- Restlet Team. (2023). RESTful API Design Best Practices. O'Reilly.
- Wilson, M. (2021). Building RESTful Web APIs with Node.js and Express. Packt Publishing.